



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Cloud Computing: Análisis,
Diseño y Despliegue de un
Servicio Empresarial Seguro
en Microsoft Azure
Documentación Técnica**



Presentado por Karima Drafi Rico
en Universidad de Burgos — Junio 2026
Tutor: D. José Ignacio Santos Martín

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	iv
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Planificación temporal	1
A.3. Estudio de viabilidad	3
Apéndice B Especificación de Requisitos	7
B.1. Introducción	7
B.2. Objetivos generales	7
B.3. Catálogo de requisitos	7
B.4. Especificación de requisitos	9
B.5. Casos de uso	10
Apéndice C Especificación de diseño	11
C.1. Introducción	11
C.2. Diseño arquitectónico	11
C.3. Diseño procedimental	12
C.4. Integración cloud	14
C.5. Diseño de datos	14
C.6. Calidad y observabilidad	14
Apéndice D Documentación técnica de programación	19

D.1. Introducción	19
D.2. Estructura de directorios	19
D.3. Ejecución local	20
D.4. Variables de entorno	20
D.5. Despliegue en Azure	20
D.6. Pruebas del sistema	20
D.7. Análisis de calidad	21
Apéndice E Documentación de usuario	23
E.1. Introducción	23
E.2. Enlaces principales de evaluación	23
E.3. Requisitos	24
E.4. Acceso al servicio	24
E.5. Operaciones disponibles	24
Apéndice F Anexo de sostenibilización curricular	27
F.1. Introducción	27
Bibliografía	31

Índice de figuras

C.1. Arquitectura cloud de la solución desplegada en Azure. Fuente: elaboración propia.	13
C.2. Grupo de recursos del proyecto en Azure Portal. Fuente: captura propia.	13
C.3. Azure App Service utilizado para alojar la API Flask y el portal web. Fuente: captura propia.	15
C.4. Azure Key Vault utilizado para almacenar el secreto de autenticación. Fuente: captura propia.	16
C.5. Identidad administrada habilitada en Azure App Service. Fuente: captura propia.	17
C.6. Storage Account utilizado para persistir solicitudes en Blob Storage. Fuente: captura propia.	18

Índice de tablas

A.1. Resumen de iteraciones del proyecto	2
A.2. Seguimiento de sprints registrado en Zube	3
A.3. Evidencias asociadas a cada sprint	4
A.4. Estimación económica del proyecto	5
A.5. Licencias y servicios utilizados	6
B.1. Requisitos con prioridad y validación	8
B.2. Casos de uso principales	10
C.1. Decisiones principales de arquitectura	12
C.2. Diccionario de datos de la solicitud	16
D.1. Pruebas realizadas sobre el prototipo	21

Apéndice A

Plan de Proyecto Software

A.1. Introducción

Este anexo describe la planificación del proyecto, el seguimiento realizado con Zube y el análisis de viabilidad del sistema.

A.2. Planificación temporal

El desarrollo del proyecto se ha organizado siguiendo un enfoque incremental basado en sprints. El seguimiento se ha realizado en Zube, utilizando historias de usuario, tareas técnicas y cambios asociados a commits del repositorio. El tablero del proyecto se encuentra en:

<https://zube.io/tfg-azure-servicio-empresarial/tfg-servicio-empresarial-seguro/w/workspace-1/kanban>

Las principales fases del proyecto han sido:

- Análisis del problema y definición de requisitos.
- Diseño de la arquitectura del sistema.
- Implementación del prototipo.
- Despliegue del sistema en la plataforma cloud.
- Validación y pruebas del sistema.

Iteraciones realizadas

La planificación se ha dividido en cinco iteraciones, superando el mínimo de tres sprints indicado en la guía docente. Cada iteración generó un incremento funcional o documental verificable:

Sprint	Objetivo	Resultado
Sprint 1 (25/02/2026–10/03/2026)	Preparación y base.	Repositorio, documentación inicial, backlog y primeras decisiones de arquitectura.
Sprint 2 (11/03/2026–24/03/2026)	Implementación de módulos y dashboard.	Clasificación ligera, portal inicial, persistencia y primeras pruebas unitarias.
Sprint 3 (25/03/2026–21/04/2026)	Incidencias y despliegue.	API de solicitudes, validación, seguridad básica y despliegue en Azure.
Sprint 4 (22/04/2026–21/05/2026)	Consolidación y documentación técnica.	App Service, Storage, Key Vault, Managed Identity, evidencias y anexos.
Sprint 5 (22/05/2026–08/07/2026)	Finalización y defensa.	Memoria, anexos, calidad SonarCloud, vídeos y entregables finales.

Tabla A.1: Resumen de iteraciones del proyecto

Seguimiento en Zube

Zube se ha utilizado como tablero Kanban para registrar historias de usuario, tareas técnicas y evolución por sprint. La tabla [A.2](#) resume el estado observable del seguimiento realizado.

Las capturas asociadas al seguimiento se conservan en la carpeta `docs/sprints/`. El Sprint 5 permanece abierto porque agrupa tareas finales de defensa y entrega oficial que se cierran una vez generados los vídeos, el PDF de enlaces y la release final.

Sprint	Inicio	Cierre/estado	Evidencia de seguimiento
Sprint 1	25/02/2026	Cerrado 15/03/2026	6 tarjetas cerradas; preparación, repositorio, arquitectura inicial y SonarCloud previsto.
Sprint 2	11/03/2026	Cerrado 17/04/2026	6 tarjetas cerradas; módulos, dashboard, IA ligera y pruebas unitarias iniciales.
Sprint 3	25/03/2026	Cerrado 20/05/2026	Integración de incidencias, seguridad, despliegue y validación de endpoints.
Sprint 4	22/04/2026	Cerrado 22/05/2026	5 tarjetas cerradas; consolidación técnica, Azure, documentación y evidencias.
Sprint 5	22/05/2026	Abierto hasta 08/07/2026	Tareas finales de memoria, anexos, vídeos, calidad, release y entrega UBUVirtual.

Tabla A.2: Seguimiento de sprints registrado en Zube

Evidencias por sprint

La tabla [A.3](#) relaciona cada iteración con las evidencias que permiten comprobar su resultado. Estas evidencias se encuentran principalmente en las carpetas `docs/sprints/` y `docs/evidencias/`.

A.3. Estudio de viabilidad

Viabilidad económica

El proyecto se ha desarrollado utilizando principalmente herramientas disponibles en versiones gratuitas, de código abierto o académicas, como GitHub, Zube y Microsoft Azure for Students. SonarCloud se utiliza como herramienta de análisis de calidad del código y su panel se aporta como evidencia de revisión externa.

Sprint	Evidencias principales	Relación con la entrega
Sprint 1	Capturas iniciales de Zube, estructura del repositorio y primeras decisiones.	Demuestra planificación inicial y definición del alcance.
Sprint 2	Progreso del tablero, primeras pruebas y evolución del portal/API.	Evidencia implementación incremental.
Sprint 3	Despliegue inicial, endpoints y validación de solicitudes.	Conecta la implementación con el despliegue cloud.
Sprint 4	Capturas de Azure Portal, Key Vault, Storage y Managed Identity.	Justifica la arquitectura cloud segura.
Sprint 5	SonarCloud, release, verificación Azure, README y entregables finales.	Aporta evidencias de cierre, calidad y defensa.

Tabla A.3: Evidencias asociadas a cada sprint

La mayor parte del coste corresponde al esfuerzo de desarrollo. Para estimar el coste de personal se ha considerado una dedicación aproximada de 300 horas y un coste de referencia de 18 euros/hora para un perfil junior de desarrollo cloud. Sobre este coste se añade una estimación del 32 % de costes empresariales asociados a Seguridad Social y otras aportaciones obligatorias. Esta cifra no representa una factura real, sino una estimación académica del valor del trabajo realizado.

En un entorno empresarial real también deberían considerarse costes de operación continuada, soporte, monitorización, almacenamiento y transferencia de datos. En este TFG se han mantenido los recursos dentro de un entorno de desarrollo de bajo consumo, por lo que el coste de infraestructura se ha reducido al mínimo. El equipo informático y el software de escritorio no se imputan por su coste completo, ya que conservan valor tras el proyecto y pueden reutilizarse en otros trabajos; en una contabilidad empresarial se imputaría únicamente su amortización proporcional al periodo de uso.

Viabilidad legal

Durante el desarrollo del proyecto se han utilizado herramientas de software bajo licencias abiertas o gratuitas para uso académico. Además, no se han utilizado datos personales reales, por lo que no se han generado problemas relacionados con la protección de datos.

Concepto	Cantidad	Coste unitario	Coste estimado
Análisis y planificación	45 h	18 EUR/h	810 EUR
Diseño de arquitectura	45 h	18 EUR/h	810 EUR
Implementación backend y portal	90 h	18 EUR/h	1.620 EUR
Infraestructura Azure y despliegue	60 h	18 EUR/h	1.080 EUR
Pruebas, documentación y memoria	60 h	18 EUR/h	1.080 EUR
Costes empresariales de personal	32 %	5.400 EUR	1.728 EUR
Servicios Azure for Students	1 suscripción	0 EUR	0 EUR
GitHub y Zube	1 cuenta académica	0 EUR	0 EUR
Total estimado	300 h		7.128 EUR

Tabla A.4: Estimación económica del proyecto

Las licencias empleadas son compatibles con un proyecto académico y permiten la distribución del código fuente del prototipo. El repositorio incluye un archivo LICENSE con licencia MIT para el código propio desarrollado durante el TFG. Las credenciales y secretos no se almacenan en el repositorio, sino en variables de entorno y Azure Key Vault.

Componente	Licencia / modalidad	Uso en el proyecto
Python	PSF License	Lenguaje backend
Flask	BSD-3-Clause	API REST y portal web
Gunicorn	MIT	Servidor WSGI en App Service
Azure SDK for Python	MIT	Integración con Storage y Key Vault
Terraform	Business Source License	Infraestructura como código
Azure CLI	Licencia Microsoft	Automatización del despliegue desde PowerShell
SonarCloud	Servicio SonarSource	Análisis de calidad del código
Microsoft Azure	Servicio cloud	App Service, Storage, Key Vault y Managed Identity

Tabla A.5: Licencias y servicios utilizados

Apéndice B

Especificación de Requisitos

B.1. Introducción

Este anexo describe los requisitos funcionales y no funcionales definidos para el sistema desarrollado.

B.2. Objetivos generales

El objetivo del sistema es permitir la gestión segura de solicitudes operativas TI utilizando servicios cloud desplegados en Microsoft Azure. Una incidencia se considera un tipo concreto de solicitud, junto con accesos, cambios de configuración, altas de aplicaciones o creación de entornos.

B.3. Catálogo de requisitos

Los requisitos funcionales recogen las capacidades visibles del sistema para usuarios, API y automatizaciones externas:

Requisitos funcionales

- RF-01 Crear solicitudes operativas TI.
- RF-02 Consultar solicitudes.
- RF-03 Obtener métricas del sistema.
- RF-04 Gestionar almacenamiento cloud.

- RF-05 Gestionar secretos mediante Azure Key Vault.
- RF-06 Clasificar solicitudes automáticamente.

Los requisitos no funcionales recogen propiedades transversales necesarias para que el sistema sea evaluable, seguro y mantenible:

Requisitos no funcionales

- RNF-01 Seguridad.
- RNF-02 Escalabilidad.
- RNF-03 Disponibilidad.
- RNF-04 Trazabilidad.

La tabla [B.1](#) amplía el catálogo de requisitos indicando prioridad y mecanismo de validación.

Código	Tipo	Prioridad	Validación
RF-01	Funcional	Alta	Prueba de <code>POST /solicitudes</code> y envío desde portal web.
RF-02	Funcional	Alta	Prueba de <code>GET /solicitudes</code> con token Bearer.
RF-03	Funcional	Media	Prueba de <code>GET /metricas</code> .
RF-04	Funcional	Alta	Comprobación de persistencia en Blob Storage y modo local.
RF-05	Funcional	Alta	Capturas de Key Vault, secreto e identidad administrada.
RF-06	Funcional	Media	Pruebas unitarias del clasificador y respuesta de creación de solicitud.
RNF-01	No funcional	Alta	Revisión de autenticación, secretos fuera del código y SonarCloud.
RNF-02	No funcional	Media	Uso de App Service y Blob Storage como servicios gestionados.
RNF-03	No funcional	Media	Endpoint <code>/health</code> y script <code>verify-azure.ps1</code> .
RNF-04	No funcional	Alta	Identificador, estado, prioridad, tipo, fecha y recomendación por solicitud.

Tabla B.1: Requisitos con prioridad y validación

B.4. Especificación de requisitos

RF-01 Crear solicitudes operativas TI

El sistema debe permitir registrar solicitudes mediante `POST /solicitudes`, validando título, descripción, tipo de solicitud y email del solicitante. El endpoint `POST /incidencias` se mantiene como compatibilidad para el caso específico de incidencias.

RF-02 Consultar solicitudes

El sistema debe listar solicitudes mediante `GET /solicitudes`, con filtros opcionales por estado, prioridad y tipo. Esta operación requiere autenticación mediante token Bearer.

RF-03 Obtener métricas

El sistema debe exponer métricas agregadas en `GET /metricas`.

RF-04 Gestionar almacenamiento cloud

Las solicitudes deben persistirse en Azure Blob Storage en entorno cloud y en fichero JSON en desarrollo local.

RF-05 Gestionar secretos mediante Azure Key Vault

El token de autenticación debe almacenarse en Key Vault y recuperarse mediante Managed Identity.

RF-06 Clasificación automática

El sistema debe clasificar automáticamente prioridad, categoría y tipo de cada solicitud, generando además una recomendación operativa.

RNF-01 Seguridad

Autenticación Bearer, almacenamiento privado y gestión de secretos centralizada.

RNF-02 Escalabilidad

Arquitectura basada en servicios gestionados de Azure App Service y Blob Storage.

RNF-03 Disponibilidad

Endpoint de salud `/health` y despliegue automatizado.

RNF-04 Trazabilidad

Registro de fecha de creación, estado, prioridad, tipo, clasificación y recomendación de cada solicitud.

B.5. Casos de uso

El sistema se ha diseñado alrededor de los siguientes casos de uso principales:

Caso de uso	Actor	Descripción
CU-01 Registrar solicitud	Usuario interno	Envía una solicitud desde el portal o mediante API REST.
CU-02 Clasificar solicitud	Sistema	Asigna tipo, prioridad, categoría y recomendación operativa.
CU-03 Consultar solicitudes	Equipo TI	Lista solicitudes y filtra por estado, tipo o prioridad.
CU-04 Consultar métricas	Equipo TI	Revisa métricas agregadas para seguimiento operativo.
CU-05 Validar despliegue	Administrador cloud	Comprueba el estado de la aplicación mediante <code>/health</code> y pruebas HTTP.
CU-06 Gestionar secretos	Administrador cloud	Mantiene tokens y configuración sensible en Azure Key Vault.

Tabla B.2: Casos de uso principales

Apéndice C

Especificación de diseño

C.1. Introducción

Este anexo describe el diseño general de la plataforma cloud para la automatización de solicitudes operativas TI.

C.2. Diseño arquitectónico

La arquitectura sigue un modelo basado en servicios cloud desplegados sobre Microsoft Azure. La aplicación se ejecuta en Azure App Service, utiliza Azure Blob Storage para persistir solicitudes, Azure Key Vault para secretos y Managed Identity para reducir la exposición de credenciales [6, 8, 7, 9].

Los principales componentes son:

- Azure App Service (API Flask y portal web).
- Azure Storage (persistencia de solicitudes).
- Azure Key Vault (gestión de secretos).
- Script PowerShell con Azure CLI para despliegue reproducible.

La tabla C.1 recoge las decisiones de arquitectura más relevantes y la alternativa principal considerada.

La figura C.1 muestra el esquema general de la solución, centrado en App Service, Blob Storage y Key Vault.

Decisión	Alternativa considerada	Justificación
App Service para la API	Máquina virtual Linux	Reduce administración de sistema operativo y facilita despliegue PaaS.
Blob Storage para persistencia	Base de datos relacional	El prototipo requiere documentos JSON sencillos y bajo coste.
Key Vault para secretos	Variables locales o fichero <code>.env</code>	Evita almacenar credenciales en el repositorio y mejora seguridad.
Managed Identity	Credenciales manuales entre servicios	Permite acceso a Key Vault sin secretos intermedios.
PowerShell + Azure CLI	Pipeline remoto obligatorio	Hace el despliegue reproducible, auditable y demostrable desde el entorno local.
Clasificador ligero	Modelo IA externo complejo	Mantiene el alcance viable y permite explicar las reglas en defensa.

Tabla C.1: Decisiones principales de arquitectura

La figura C.2 recoge una evidencia del grupo de recursos utilizado durante el despliegue del prototipo.

C.3. Diseño procedimental

El flujo principal del sistema es:

1. Recepción de solicitudes operativas TI por portal web o API REST.
2. Validación de campos obligatorios.
3. Clasificación automática mediante componente de IA ligera.
4. Generación de recomendaciones de actuación operativa.

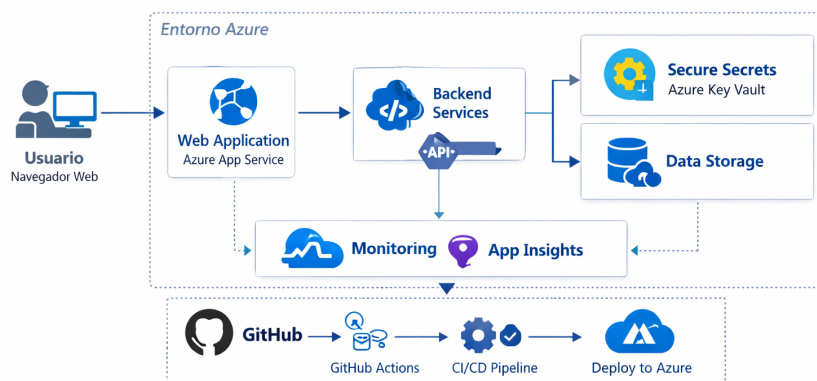


Figura C.1: Arquitectura cloud de la solución desplegada en Azure. Fuente: elaboración propia.

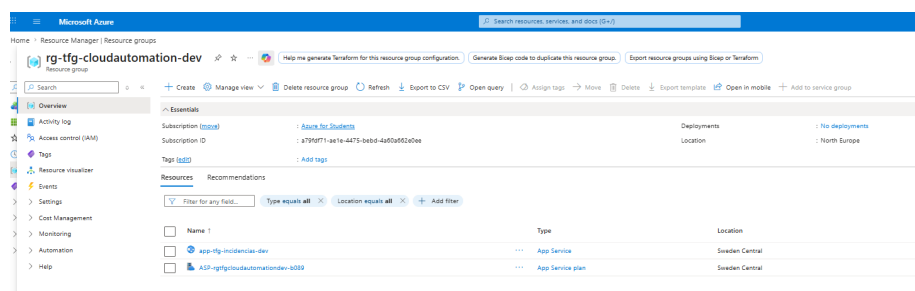


Figura C.2: Grupo de recursos del proyecto en Azure Portal. Fuente: captura propia.

5. Persistencia de solicitudes en Blob Storage o fichero local.
6. Respuesta JSON con identificador, prioridad, tipo, recomendación y confianza.
7. Verificación del despliegue mediante endpoint `/health`, portal web y pruebas HTTP.

Tipos de solicitud soportados

- Acceso a recursos (VPN, permisos, cuentas).
- Provisión de entornos (desarrollo, staging, producción).
- Alta de aplicaciones o servicios.
- Cambios de configuración.
- Incidencias o peticiones de soporte.

C.4. Integración cloud

- **Terraform**: documenta Resource Group, App Service, Storage y Key Vault como infraestructura declarativa.
- **Bicep**: plantillas alternativas para reproducir la infraestructura.
- **Script PowerShell**: configura recursos existentes y publica la aplicación en Azure App Service.

Las figuras [C.3](#), [C.4](#), [C.5](#) y [C.6](#) muestran evidencias de los principales servicios utilizados.

C.5. Diseño de datos

La persistencia del prototipo utiliza un documento JSON por solicitud en Azure Blob Storage. No existe una base de datos relacional, por lo que el modelo de datos se describe como una entidad documental.

C.6. Calidad y observabilidad

La calidad se comprueba mediante pruebas automáticas y análisis manual en SonarCloud. El panel de SonarCloud se aporta como evidencia de mantenibilidad, fiabilidad, seguridad y duplicidad del código. La observabilidad técnica se apoya en el endpoint `/health`, en los logs de App Service y en las métricas disponibles desde Azure Monitor [\[5\]](#).

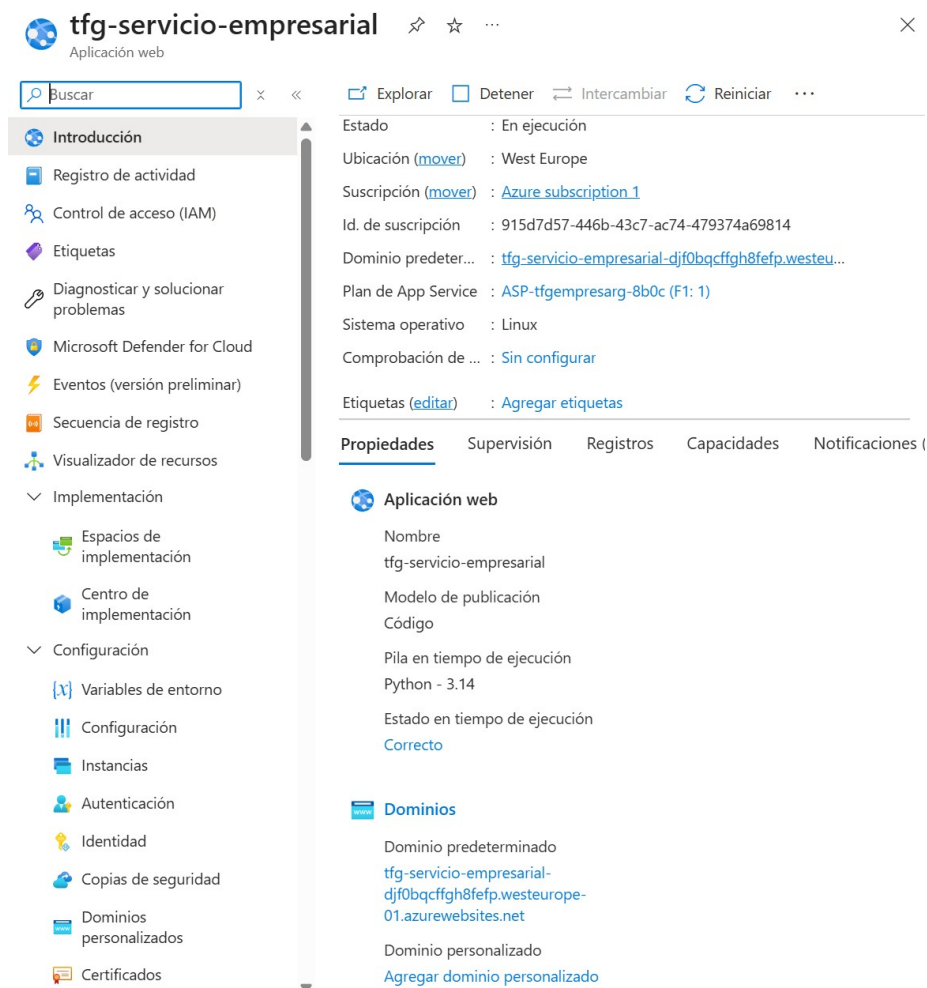


Figura C.3: Azure App Service utilizado para alojar la API Flask y el portal web. Fuente: captura propia.

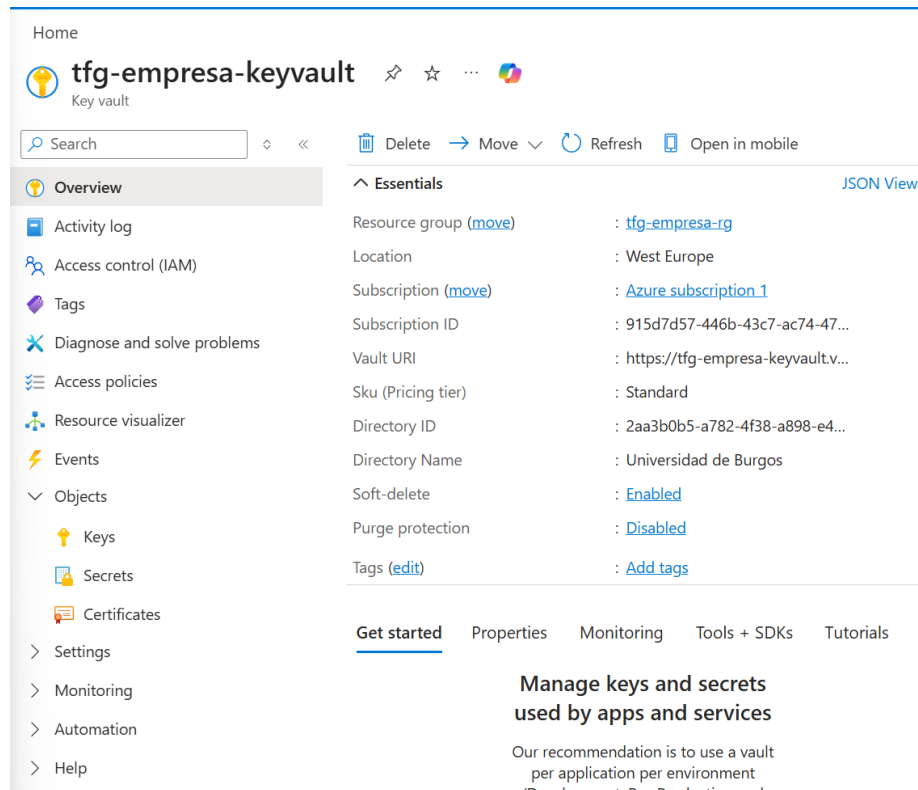


Figura C.4: Azure Key Vault utilizado para almacenar el secreto de autenticación. Fuente: captura propia.

Campo	Tipo	Descripción
id	texto	Identificador único de la solicitud.
titulo	texto	Resumen de la petición enviada por el usuario.
descripcion	texto	Detalle de la solicitud o incidencia.
tipo_solicitud	texto	Acceso, entorno, aplicación, configuración o incidencia.
prioridad	texto	Prioridad calculada por el clasificador.
categoria	texto	Área funcional detectada: infraestructura, seguridad, software u otra.
estado	texto	Estado operativo de la solicitud.
recomendacion	texto	Acción sugerida para el equipo TI.
reportado_por	texto	Correo o identificador del solicitante.
created_at	fecha	Fecha de creación de la solicitud.

Tabla C.2: Diccionario de datos de la solicitud

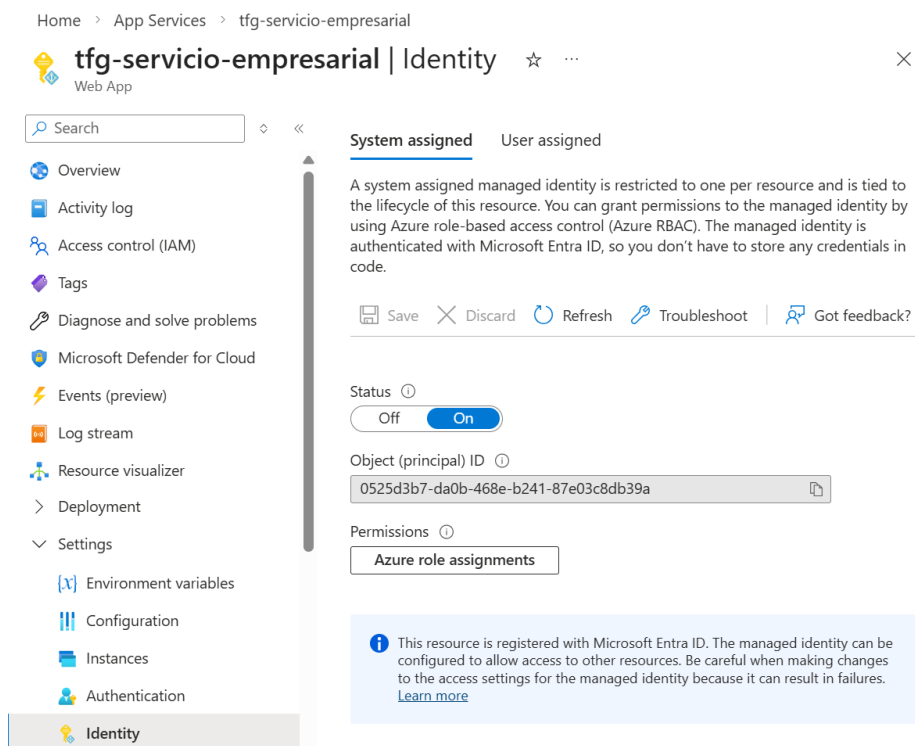


Figura C.5: Identidad administrada habilitada en Azure App Service. Fuente: captura propia.

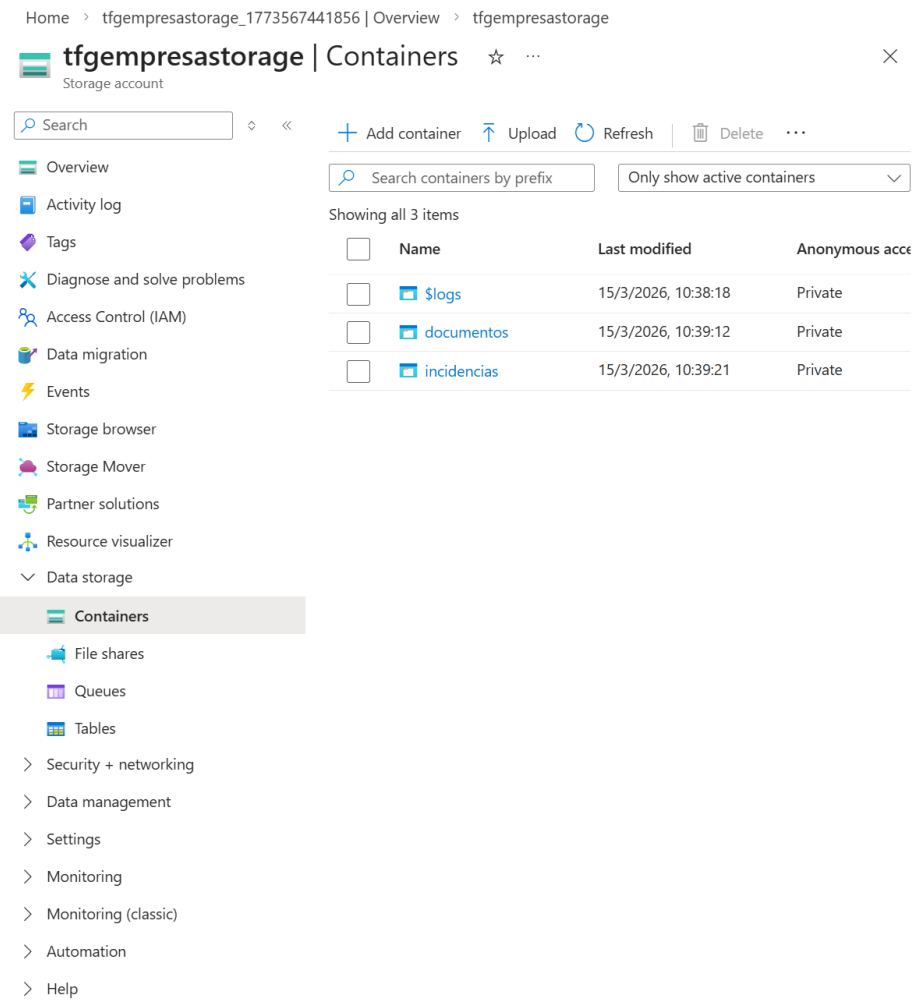


Figura C.6: Storage Account utilizado para persistir solicitudes en Blob Storage. Fuente: captura propia.

Apéndice D

Documentación técnica de programación

D.1. Introducción

Este anexo describe la estructura técnica del repositorio y los pasos para ejecutar, probar y desplegar la solución, apoyándose en scripts reproducibles con Azure CLI y servicios gestionados de Azure [4, 9].

D.2. Estructura de directorios

- `src/app.py`: API principal Flask.
- `src/classifier.py`: clasificador automático de incidencias.
- `src/storage.py`: persistencia local o en Azure Blob Storage.
- `src/config.py`: configuración por variables de entorno.
- `src/requirements.txt`: dependencias Python.
- `tests/test_api.py`: pruebas unitarias.
- `infra/terraform/`: infraestructura como código.
- `scripts/`: despliegue y verificación en Azure.
- `memoria/tex/`: memoria y anexos en LaTeX.

D.3. Ejecución local

```
cd src
pip install -r requirements.txt
set STORAGE_MODE=local
python app.py
```

La API quedará disponible en `http://localhost:5000`.

D.4. Variables de entorno

- `STORAGE_MODE`: `local` o `azure`.
- `AZURE_STORAGE_CONNECTION_STRING`: cadena de conexión del Storage Account.
- `KEY_VAULT_URL`: URI del Key Vault.
- `API_KEY`: token Bearer opcional para desarrollo local.

D.5. Despliegue en Azure

1. Iniciar sesión con `az login`.
2. Ejecutar `scripts/deploy-azure.ps1` desde la raíz del repositorio.
3. Comprobar `/health` y abrir el portal web desplegado.
4. Ejecutar `scripts/verify-azure.ps1` para verificar endpoints principales.

D.6. Pruebas del sistema

```
python -m unittest discover -s tests -p "test_*.py"
```

Las pruebas verifican endpoints, creación de incidencias, métricas y clasificación automática.

La tabla [D.1](#) resume las pruebas principales aplicadas durante la validación del prototipo.

Prueba	Objetivo	Resultado esperado
Endpoint raíz	Comprobar disponibilidad básica del servicio.	Respuesta 200 con estado operacional.
Creación de solicitud	Validar entrada, clasificación y persistencia.	Respuesta 201 con identificador, prioridad y recomendación.
Portal web	Verificar acceso a la interfaz de usuario.	HTML del portal cargado correctamente.
Métricas	Comprobar agregación de solicitudes.	Totales por estado, prioridad y tipo.
Clasificación de seguridad	Validar detección de incidencias críticas.	Categoría seguridad y prioridad alta.
Verificación Azure	Probar despliegue real con script.	Health check, creación, listado y métricas correctas.
SonarCloud	Revisar calidad externa del código.	Quality Gate aprobado y duplicación controlada.

Tabla D.1: Pruebas realizadas sobre el prototipo

D.7. Análisis de calidad

El proyecto utiliza SonarCloud como servicio gestionado de SonarQube:

- Panel del proyecto: https://sonarcloud.io/project/overview?id=karimadrico_tfg-azure-servicio-empresarial-seguro.
- docs/calidad/sonarcloud.md: documenta el resultado observado y las evidencias que deben guardarse.

Para completar la evidencia de calidad se debe guardar una captura del Quality Gate, una captura del resumen de métricas y la URL del panel en el documento final de enlaces.

Apéndice *E*

Documentación de usuario

E.1. Introducción

Este anexo describe cómo utilizar la plataforma de gestión de solicitudes operativas TI desplegada en Microsoft Azure.

E.2. Enlaces principales de evaluación

Los siguientes enlaces permiten acceder directamente a los recursos principales que debe revisar el tribunal:

- **Portal web desplegado**
<https://app-tfg-incidencias-dev-fme6drcgg6bwenbg.swedencentral-01.azurewebsites.net/portal>
- **Repositorio GitHub**
<https://github.com/karimadrico/tfg-azure-servicio-empresarial-seguro>
- **Tablero Zube utilizado para la planificación por sprints**
<https://zube.io/tfg-azure-servicio-empresarial/tfg-servicio-empresarial-seguro/w/workspace-1/kanban>
- **Panel SonarCloud utilizado para el seguimiento de calidad del código**
https://sonarcloud.io/project/overview?id=karimadrico_tfg-azure-servicio-empresarial-seguro

E.3. Requisitos

El usuario necesita:

- Cliente HTTP (navegador, Postman o curl).
- Conexión a Internet.
- Token de autenticación Bearer (excepto en entorno local sin clave configurada).

E.4. Acceso al servicio

La API está desplegada en Azure App Service:

`https://app-tfg-incidencias-dev-fme6drcgg6bwenbg.swedencentral-01.azurewebsites.net`

El portal web está disponible en:

`https://app-tfg-incidencias-dev-fme6drcgg6bwenbg.swedencentral-01.azurewebsites.net/portal`

E.5. Operaciones disponibles

Consultar estado del servicio

GET /

Crear solicitud desde el portal

El usuario puede abrir el portal web, seleccionar el tipo de solicitud, introducir título, descripción y correo del solicitante, y enviar el formulario. La API devolverá un identificador, prioridad, clasificación y recomendación operativa.

Crear solicitud desde API

POST /solicitudes
Content-Type: application/json

```
{
  "tipo_solicitud": "acceso",
  "titulo": "Acceso VPN",
  "descripcion": "Necesito acceso VPN al entorno cloud",
  "reportado_por": "usuario@empresa.com",
  "prioridad": "media"
}
```

El sistema clasificará automáticamente la solicitud y devolverá prioridad, categoría, tipo, nivel de confianza y recomendación.

Consultar solicitudes

```
GET /solicitudes?estado=abierta&prioridad=alta&tipo_solicitud=acceso
Authorization: Bearer <token>
```

Si se accede a esta URL directamente desde el navegador sin cabecera de autorización, la API responde `{.error:"No autenticado"}`, que es el comportamiento esperado para proteger el listado.

Consultar métricas

```
GET /metricas
Authorization: Bearer <token>
```

Anexo de sostenibilización curricular

F.1. Introducción

El desarrollo de soluciones cloud puede contribuir significativamente a mejorar la sostenibilidad tecnológica y organizativa cuando se diseña con criterios de eficiencia, seguridad y responsabilidad [1, 2]. En este proyecto se ha desarrollado una plataforma para automatizar solicitudes operativas TI en Microsoft Azure, evitando un enfoque basado en procesos manuales, correos dispersos o documentación no estructurada. Esta orientación se relaciona con varios Objetivos de Desarrollo Sostenible, especialmente con el ODS 9, industria, innovación e infraestructura; el ODS 12, producción y consumo responsables; y el ODS 13, acción por el clima.

Desde el punto de vista del ODS 9, la solución propone una infraestructura digital moderna basada en servicios gestionados. El uso de Azure App Service, Storage, Key Vault y Managed Identity permite construir un sistema escalable sin necesidad de adquirir, instalar y mantener servidores físicos propios. La infraestructura como código mediante Terraform y Bicep y los scripts de despliegue favorecen la reproducibilidad, reducen errores de configuración y permiten que el entorno pueda recrearse de forma controlada, en línea con buenas prácticas de arquitectura cloud [3]. Esto contribuye a una innovación más sostenible, ya que el conocimiento generado no queda limitado a una configuración manual difícil de auditar.

En relación con el ODS 12, el proyecto fomenta un uso responsable de recursos tecnológicos. La aplicación se ha desplegado en un entorno

de bajo consumo, ajustado al alcance académico y sin sobredimensionar la infraestructura. La plataforma cloud permite activar únicamente los recursos necesarios para validar el prototipo, evitando gastos innecesarios y reduciendo el desperdicio tecnológico. Además, la automatización de despliegues y verificaciones disminuye la repetición de tareas manuales y reduce la probabilidad de errores humanos, lo que mejora la eficiencia operativa.

El ODS 13 también está presente de forma indirecta. Aunque cualquier servicio digital consume energía, el uso de proveedores cloud permite beneficiarse de centros de datos optimizados, con mejores tasas de utilización que muchas infraestructuras locales pequeñas. En lugar de mantener servidores dedicados infrautilizados, el proyecto usa servicios compartidos y escalables. Esta decisión no elimina el impacto ambiental, pero sí representa una alternativa más eficiente para un prototipo académico y para muchas organizaciones pequeñas que necesitan digitalizar procesos internos.

La sostenibilidad también tiene una dimensión social y organizativa. La plataforma mejora la trazabilidad de solicitudes internas, facilita priorizar incidencias críticas, registra recomendaciones operativas y permite consultar métricas del servicio. Esto puede ayudar a equipos técnicos a trabajar con mayor transparencia y a reducir tiempos de respuesta. Un proceso más ordenado también evita duplicidades, pérdidas de información y dependencia excesiva de conocimiento informal.

Por último, el proyecto incorpora criterios de seguridad y protección de la información. El uso de Key Vault, Managed Identity y autenticación Bearer evita almacenar secretos directamente en el código fuente, siguiendo recomendaciones de diseño seguro en Azure [10]. Esta práctica favorece un desarrollo responsable y reduce riesgos asociados a fugas de credenciales. Aunque el sistema utiliza datos de prueba y no datos personales reales, se han aplicado principios básicos de minimización y protección.

Otra aportación relevante es la mantenibilidad del sistema. La solución se documenta mediante memoria, anexos, guías de despliegue y scripts reproducibles, de forma que otra persona pueda revisar el diseño, ejecutar pruebas y desplegar el prototipo sin depender únicamente de conocimiento verbal. Esta trazabilidad técnica reduce el riesgo de abandono del software tras la entrega académica y facilita que el trabajo pueda evolucionar hacia nuevas funciones, como autenticación corporativa, mejoras de observabilidad o integración con más flujos empresariales.

En conjunto, el TFG demuestra que una solución cloud académica puede diseñarse no solo para funcionar, sino también para ser mantenible, segura,

eficiente y alineada con criterios de sostenibilidad curricular. La principal aportación no es únicamente el software desarrollado, sino la aplicación de buenas prácticas profesionales para automatizar procesos internos con un impacto razonable y controlado.

Bibliografía

- [1] CRUE Universidades Españolas. Directrices de sostenibilidad en trabajos de fin de grado y proyectos, 2012. Consultado: marzo 2026.
- [2] Microsoft Azure. Microsoft cloud adoption framework for azure, 2024. Guía oficial de buenas prácticas de adopción de nube de Microsoft Learn para entornos Azure, consultado marzo 2026.
- [3] Microsoft Azure. Azure well-architected framework, 2025. Marco de mejores prácticas para diseño, fiabilidad, seguridad y costes en arquitecturas cloud de Azure, consultado marzo 2026.
- [4] Microsoft Learn. Azure key vault documentation, 2024. Consultado: marzo 2026.
- [5] Microsoft Learn. Azure monitor documentation, 2024. Consultado: marzo 2026.
- [6] Microsoft Learn. Configure a linux python app for azure app service, 2026. Documentación oficial de Microsoft Learn, consultado junio 2026.
- [7] Microsoft Learn. Managed identities for azure app service, 2026. Documentación oficial de Microsoft Learn, consultado junio 2026.
- [8] Microsoft Learn. Quickstart: Azure blob storage client library for python, 2026. Documentación oficial de Microsoft Learn, consultado junio 2026.
- [9] Microsoft Learn. Tutorial: Python app service using key vault and managed identity, 2026. Documentación oficial de Microsoft Learn, consultado junio 2026.

- [10] Microsoft Learn. Área de diseño: Seguridad - cloud adoption framework, 2026. Guía de diseño de seguridad en Azure (Cloud Adoption Framework), consultado marzo 2026.